

MEMORIA TFG - ApVet

Juan Pérez Medina

Índice

1. Análisis del problema.....	1
1.1. Introducción.....	1
1.2. Objetivos.....	1
1.3. Funciones y rendimientos deseados.....	1
1.4. Planteamiento y evaluación de diversas soluciones.....	1
1.5. Justificación de la solución elegida.....	1
1.6. Modelado de la solución.....	2
1.6.1. Recursos humanos.....	2
1.6.2. Recursos hardware.....	2
1.6.3. Recursos software.....	2
1.7. Planificación temporal.....	2
2. Diseño e implementación del proyecto.....	2
2.1. Arquitectura del sistema.....	2
2.2. Diseño de la base de datos.....	2
2.3. Mapa de navegación.....	3
2.4. Diseño de páginas.....	3
2.5. Despliegue y Seguridad de la Aplicación.....	3
3. Fase de pruebas.....	3
4. Documentación de la Aplicación.....	4
4.1. Introducción a la Aplicación.....	4
4.2. Manual de Instalación.....	4
4.3. Manual de Usuario.....	4
4.4. Manual de Administración.....	4
5. Resumen y capturas.....	5
6. Conclusiones finales.....	7
7. Bibliografía.....	7

1. Análisis del problema

1.1. Introducción

El sector veterinario requiere de un suministro constante y seguro de productos fitosanitarios y material especializado. ApVet nace como una plataforma B2B (Business-to-Business) orientada a modernizar y agilizar el proceso de compra de productos veterinarios para clínicas y colegiados.

1.2. Objetivos

- Desarrollar una tienda online exclusiva para veterinarios colegiados.
- Garantizar la seguridad y privacidad del catálogo mediante un sistema de autenticación restringido.
- Implementar un panel de administración robusto para gestionar productos, usuarios y pedidos.
- Diseñar una interfaz de usuario (UI) moderna, rápida y adaptable (responsive).
- Simplificar el proceso de facturación y pedido mostrando únicamente tarifas profesionales.

1.3. Funciones y rendimientos deseados

El sistema debe permitir a los veterinarios consultar un catálogo de productos, añadirlos a un carrito y realizar pedidos de forma ágil. Los administradores deben poder dar de alta a nuevos usuarios (vetando el registro público), gestionar el stock, crear productos con imágenes y modificar el estado de los pedidos (Pendiente, En proceso, Finalizado, Cancelado). En términos de rendimiento, se exige carga instantánea de vistas mediante una SPA (Single Page Application) y tiempos de respuesta de la API muy reducidos.

1.4. Planteamiento y evaluación de diversas soluciones

Solución elegida: Arquitectura Desacoplada (Frontend en React + Backend API REST). Permite total libertad en el diseño de la UI y separar claramente las responsabilidades del servidor.

1.5. Justificación de la solución elegida.

Se optó por la arquitectura desacoplada. Usar React (con Vite y TypeScript) en el frontend permite crear una experiencia de usuario inmersiva y sin recargas de página. En el backend, Python con Flask y SQLAlchemy ofrece una creación de API ágil y un manejo de base de datos relacional seguro. Para la base de datos se eligió MySQL paquetizado en Docker para aislar el entorno y facilitar el despliegue.

Todo muy sencillo a la vez que eficaz.

1.6. Modelado de la solución

1.6.1. Recursos humanos

El proyecto ha sido desarrollado de forma individual, asumiendo los roles de Analista de Requisitos, Diseñador UX/UI, Desarrollador Frontend y Backend, y Administrador de Sistemas.

1.6.2. Recursos hardware

- Equipo de desarrollo: PC estándar (16 GB de RAM) con Sistema Operativo Windows 10.
- Servidor de despliegue: Capacidad para ejecutar contenedores Docker.

1.6.3. Recursos software

- Entorno de desarrollo: Visual Studio Code, Docker Desktop, Node.js y Python 3.

1.7. Planificación temporal

El proyecto se ha dividido en iteraciones (sprints):

- Fase 1: Toma de requisitos y diseño de base de datos.
- Fase 2: Desarrollo del Backend y de la API RESTful.
- Fase 3: Desarrollo del Frontend (Maquetación y vistas principales).
- Fase 4: Integración Front-Back y programación del panel de administración.
- Fase 5: Pruebas, corrección de errores (debugging) y redacción de la documentación.

2. Diseño e implementación del proyecto

2.1. Arquitectura del sistema

Se trata de una arquitectura cliente-servidor basada en microservicios ligeros.

- Cliente: Aplicación React compilada con Vite.
- Servidor: API RESTful programada en Flask que expone endpoints en formato JSON.
- Base de datos: MySQL ejecutándose en un contenedor Docker con persistencia a través de volúmenes.

La comunicación se realiza vía HTTP usando tokens JWT para la autorización y seguridad.

2.2. Diseño de la base de datos

Se utiliza una base de datos relacional MySQL (`veterinaria_db`) estructurada y normalizada en 6 tablas principales:

- roles (id, nombre)
- usuarios (id, nombre, apodo, telefono, password, num_colegiado, rol_id)

- categorias (id, nombre)
- productos (id, nombre, descripcion, precio_profesional, stock, imagen, oculto, categoria_id)
- pedidos (id, usuario_id, fecha, estado, total)
- detalle_pedido (id, pedido_id, producto_id, cantidad, precio_unitario_aplicado)

2.3. Mapa de navegación

- Visitante no logueado: Pantalla de Login exclusivamente.
- Veterinario: Catálogo -> Detalles de Producto -> Carrito -> Mi Perfil (Mis Datos, Mis Pedidos) -> Sobre Nosotros
- Administrador: Tiene acceso a todas las rutas anteriores y, además, a las herramientas de gestión (Gestión de Productos, Gestión de Usuarios, Gestión de Pedidos) ubicadas en Mi Perfil.

2.4. Diseño de páginas

El diseño se ha realizado siguiendo las directrices de diseño moderno y la estética corporativa del sector salud. Se ha utilizado Tailwind CSS para la maquetación rápida y responsiva.

Las vistas principales son:

- Login: Formulario centrado, limpio y minimalista.
- Catálogo: Cuadrícula de tarjetas de productos con un buscador en tiempo real y filtros dinámicos en la parte superior.
- Perfil: Sistema de pestañas para navegar fluidamente entre Datos Personales, Historial de compras y herramientas de gestión.

2.5. Despliegue y Seguridad de la Aplicación

- Despliegue: Orquestado mediante Docker Compose, automatizando el levantamiento de la base de datos MySQL.
- Seguridad: Cifrado de contraseñas con el algoritmo bcrypt. Protección de todas las rutas en Frontend y Backend mediante tokens JWT con expiración de 24h. Bloqueo de registro público (las cuentas son creadas manualmente por el administrador para garantizar que solo acceden colegiados reales).

3. Fase de pruebas

Se realizaron pruebas unitarias manuales y pruebas de integración exhaustivas.

- Pruebas funcionales: Comprobación del ciclo completo de compra, validando la inserción del pedido en la base de datos y el descuento correcto del stock.
- Pruebas de seguridad: Intento de acceso a rutas protegidas sin proporcionar token, y validación contra ataques de inyección SQL.

- Pruebas UI: Verificación del comportamiento responsivo en distintos tamaños de pantalla (móvil, tablet, escritorio) garantizando una experiencia de usuario perfecta.

4. Documentación de la Aplicación.

4.1. Introducción a la Aplicación

ApVet es un sistema integral B2B. Al acceder, el sistema bloquea cualquier interacción no autorizada presentando directamente la pasarela de inicio de sesión. Esto asegura la privacidad del catálogo y los precios frente al público general.

4.2. Manual de Instalación

1. Clonar el repositorio del proyecto.
2. Levantar la base de datos ejecutando: ``docker-compose up -d``
3. Arrancar Backend: Desde la carpeta ``/backend``, instalar dependencias con ``pip install -r requirements.txt`` y ejecutar ``python src/app.py``.
4. Arrancar Frontend: Desde la carpeta ``/frontend``, ejecutar ``npm install`` para descargar dependencias y luego ``npm run dev`` para iniciar el servidor.

4.3. Manual de Usuario

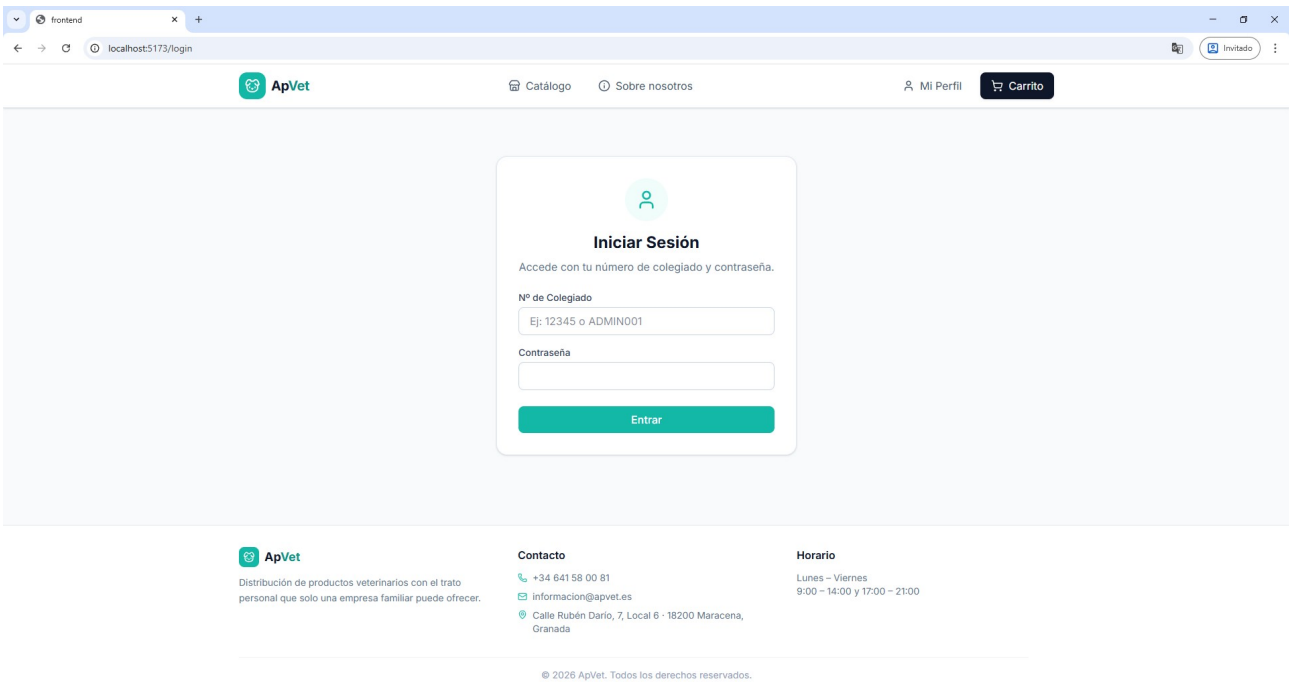
- Acceso: Ingresar el Número de Colegiado en el campo usuario y la contraseña asignada.
- Comprar: Navegar por el catálogo, buscar productos o filtrar por categorías. Usar el botón "Añadir al Carrito" y confirmar la compra en la pantalla "Carrito".
- Perfil: Acceder al menú "Mi Perfil" para cambiar teléfono, apodo, renovar la contraseña, o consultar el estado de facturación de pedidos anteriores.

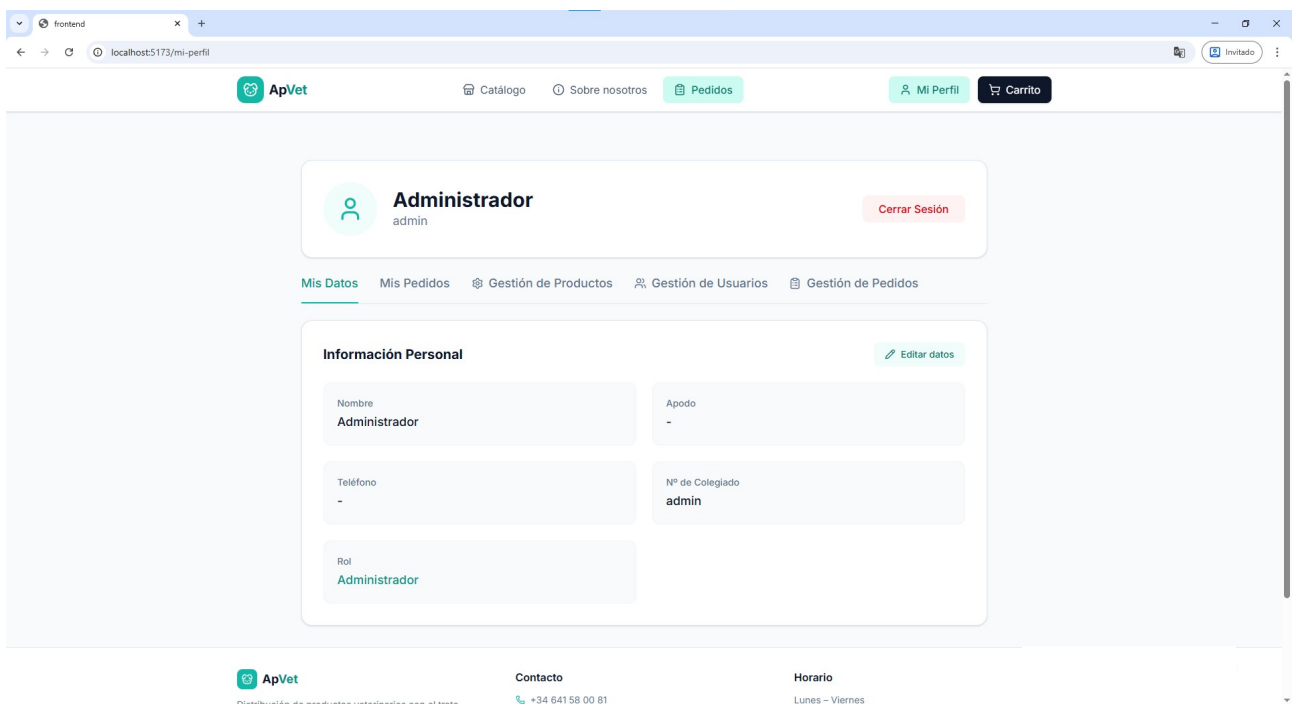
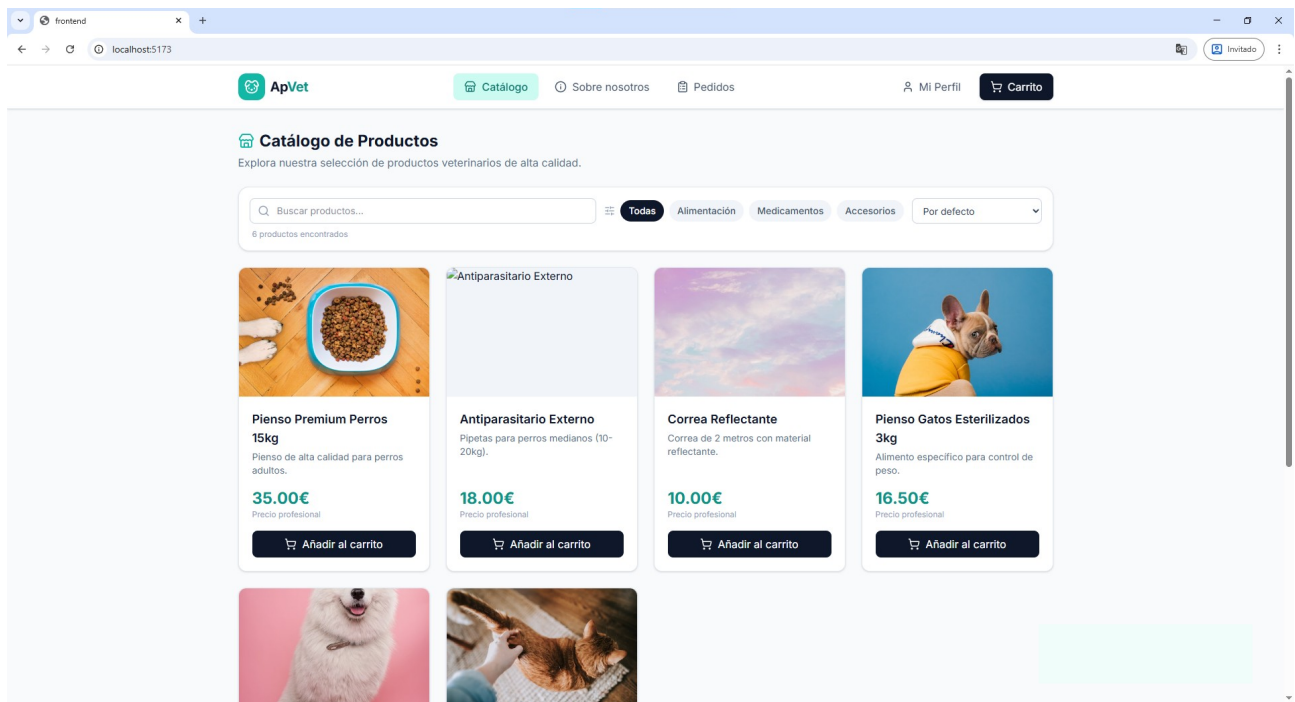
4.4. Manual de Administración

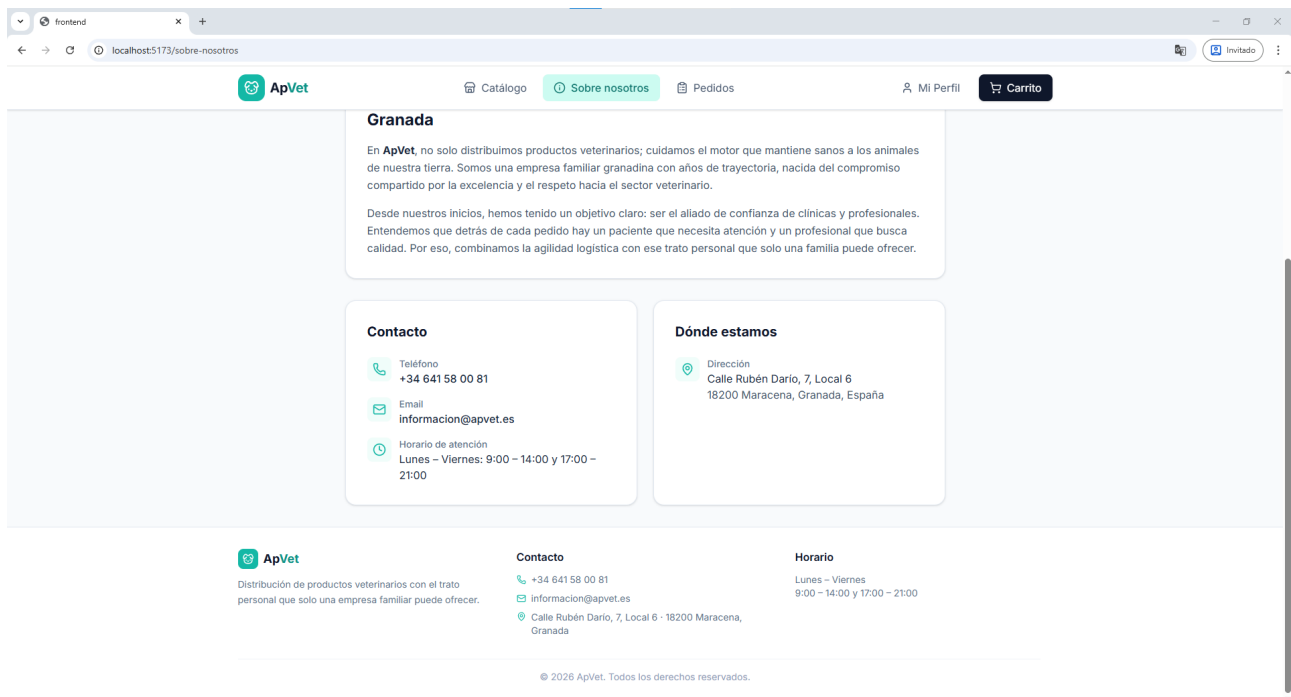
El administrador dispone de las siguientes funcionalidades de control en su panel:

- Gestión de Productos: Dar de alta nuevos artículos, subir sus fotografías, asignarles una categoría o marcarlos como ocultos si se retiran del catálogo temporalmente.
- Gestión de Usuarios: Formulario para registrar nuevos veterinarios en la plataforma.
- Gestión de Pedidos: Tablero para visualizar todos los pedidos entrantes de cualquier usuario y cambiar su estado logístico (Pendiente -> En proceso -> Finalizado/Cancelado). Cuenta con un sistema de notificaciones en vivo en la barra de navegación para pedidos sin atender.

5. Resumen y capturas







6. Conclusiones finales

El desarrollo de ApVet ha permitido consolidar conocimientos de desarrollo Fullstack, abarcando desde la planificación inicial y el modelado relacional de la base de datos hasta el empaquetado en contenedores Docker. Se ha logrado una aplicación rápida, segura y altamente escalable. La decisión de separar Front y Back (React + Flask) ha demostrado ser un acierto absoluto, facilitando la mantenibilidad, el desarrollo paralelo y la detección temprana de errores.

7. Bibliografía

Documentación oficial de React (<https://react.dev/>)

Documentación oficial de Vite (<https://vitejs.dev/>)

Documentación oficial de Flask (<https://flask.palletsprojects.com/>)

Documentación de Tailwind CSS (<https://tailwindcss.com/>)

Documentación de Docker (<https://docs.docker.com/>)

Guías de SQLAlchemy (<https://www.sqlalchemy.org/>)